

Домашнее задание №5

Цель: получить практический опыт установки пакетов с помощью сторонних репозиторий и пакетного менеджера. Научиться выполнять задачи автоматизации с помощью Bash.

Задание 1 – добавить в cron скрипт/команду, которая будет очищать кэш apt (кэшируемые пакеты, пакеты, которые не могут быть загружены) раз в месяц в 16 часов.

Опционально:

Задание 2 – в соответствии с нижеуказанным примером, запустить демон nodejs-приложения через systemd.

Запуск демона приложения через systemd

Серверные приложения, написанные, к примеру, на nodejs или на python, держат всегда запущенными. Если приложение вдруг упало, нужно его

Для примера попробуем запустить такое приложение, написанное на nodejs:

```
const http = require('http');

// Get MYAPP_PORT from environment variable
const MYAPP_PORT = process.env.MYAPP_PORT;

http.createServer((req, res) => {
  if (req.url === '/kill') {
    // App die on uncaught error and print stack trace to stderr
    throw new Error('Someone kills me');
  }

  if (req.method === 'POST') {
    // App print this message to stderr, but is still alive
    console.error(`Error: Request ${req.method} ${req.url}`);

    res.writeHead(405, { 'Content-Type': 'text/plain' });
    res.end('405 Method Not Allowed');

    return;
  }
})
```

```
// App print this message to stdout
console.log(`Request ${req.method} ${req.url}`);

res.writeHead(200, { 'Content-Type': 'text/plain' });
res.end('200 OK');
})
.listen(MYAPP_PORT);
```

Этот скрипт запускает веб-сервер. Он слушает порт, который задан в переменной окружения MYAPP_PORT. Запущу приложение, чтобы проверить:

```
MYAPP_PORT=3000 node index.js
```

Приложение на все гет-запросы отвечает «200 OK»:

```
curl 'http://localhost:3000'
```

На пост-запросы отвечает «405 Method Not Allowed»:

```
curl -X POST 'http://localhost:3000'
```

И при запросе /kill вообще умирает:

```
curl 'http://localhost:3000/kill'
```

Сервис-юнит

Systemd оперирует понятиями *юнитов*. Юнитом может быть процесс-демон, устройство, или даже список других юнитов. В руководстве `man systemd` в секции «Concepts» описано, какие есть типы юнитов, и для чего они нужны.

Мне понадобится самый первый — Service unit. Он позволяет запустить процесс в режиме демона и перезапускать процесс, если он упал. В руководстве `man systemd.unit` описано, как писать юниты в целом, а в `man systemd.service` — как писать сервис-юниты.

Файлы своих юнитов нужно класть в `/etc/systemd/system/`. Я назову свой `/etc/systemd/system/myapp.service`. Минимальная конфигурация для сервиса может быть такая:

```
[Service]
```

```
Environment=MYAPP_PORT=3000
ExecStart=/usr/local/bin/node /usr/local/www/myapp/index.js
```

В поле Environment нужны приложению переменные окружения.
В ExecStart команда для запуска приложения. /usr/local/bin/node — это полный путь к nodejs, его можно узнать командой:

which node

А /usr/local/www/myapp/index.js — полный путь к моему приложению.

Пробую запустить сервис:

```
sudo systemctl start myapp.service
```

Проверю статус:

```
sudo systemctl status myapp.service
```

```
● myapp.service
  Loaded: loaded (/etc/systemd/system/myapp.service; static; vendor preset:
  enabled)
  Active: active (running) since Fri 2017-08-04 21:50:14 MSK; 5s ago
  Main PID: 3815 (node)
    Tasks: 6
   Memory: 7.7M
      CPU: 80ms
  CGroup: /system.slice/myapp.service
          └─3815 /usr/local/bin/node /usr/local/www/myapp/index.js
```

```
Aug 04 21:50:14 ubuntu16 systemd[1]: Started myapp.service.
```

Ну типа работает. Дёрну курлом, чтобы проверить:

```
curl 'http://localhost:3000/'
200 OK
```

Действительно, работает.

Автоматический перезапуск

Если приложение по какой-то причине умрёт, то оно не поднимется. Вот я дёргаю убивающий эндпойнт:

```
curl 'http://localhost:3000/kill'
curl: (52) Empty reply from server
```

И проверяю статус приложения:

```
sudo systemctl status myapp.service
```

```
● myapp.service
```

```
Loaded: loaded (/etc/systemd/system/myapp.service; static; vendor preset:
enabled)
```

```
Active: failed (Result: exit-code) since Fri 2017-08-04 21:51:46 MSK; 5s ago
```

```
Process: 3815 ExecStart=/usr/local/bin/node /usr/local/www/myapp/index.js
(code=exited, status=1/FA
```

```
Main PID: 3815 (code=exited, status=1/FAILURE)
```

```
Aug 04 21:51:46 ubuntu16 node[3815]:      ^
Aug 04 21:51:46 ubuntu16 node[3815]: Error: Someone kills me
Aug 04 21:51:46 ubuntu16 node[3815]:   at Server.http.createServer
(/usr/local/www/myapp/index.js:1
Aug 04 21:51:46 ubuntu16 node[3815]:   at emitTwo (events.js:106:13)
Aug 04 21:51:46 ubuntu16 node[3815]:   at Server.emit (events.js:191:7)
Aug 04 21:51:46 ubuntu16 node[3815]:   at HTTPParser.parserOnIncoming [as
onIncoming] (_http_server
Aug 04 21:51:46 ubuntu16 node[3815]:   at
HTTPParser.parserOnHeadersComplete (_http_common.js:99:23
Aug 04 21:51:46 ubuntu16 systemd[1]: myapp.service: Main process exited,
code=exited, status=1/FAILUR
Aug 04 21:51:46 ubuntu16 systemd[1]: myapp.service: Unit entered failed state.
Aug 04 21:51:46 ubuntu16 systemd[1]: myapp.service: Failed with result
'exit-code'.
```

Чтобы systemd переподнимал упавшее приложение, нужно добавить опцию Restart. У неё есть несколько значений, подробно описанных в man systemd.service. Значение always будет в любой ситуации перезапускать приложение:

```
[Service]
```

```
Environment=MYAPP_PORT=3000
```

```
ExecStart=/usr/local/bin/node /usr/local/www/myapp/index.js
```

```
Restart=always
```

Когда изменили файл юнита, надо заставить systemd перечитать его, чтобы он использовал уже новый файл:

```
sudo systemctl daemon-reload
```

Снова запущу сервис, дёрну убивающую ручку и проверю, переподнимется ли приложение:

```
sudo systemctl start myapp.service
```

```
curl 'http://localhost:3000/kill'  
curl: (52) Empty reply from server
```

```
curl 'http://localhost:3000/'  
200 OK
```

```
curl 'http://localhost:3000/kill'  
curl: (52) Empty reply from server
```

```
curl 'http://localhost:3000/'  
200 OK
```

Ну вот. Теперь упавший процесс переподнимается.

Ограничение прав

Пользователя и группу, от имени которых будет запущен процесс, можно установить переменными User и Group:

```
[Service]  
Environment=MYAPP_PORT=3000  
User=www-data  
Group=www-data  
ExecStart=/usr/local/bin/node /usr/local/www/myapp/index.js  
Restart=always
```

Запись и просмотр логов

Здорово, если ваше приложение пишет логи в файлы. Но если оно пишет в стандартный вывод, то через systemd можно собирать эти записи. По умолчанию они и так куда-то собираются. Куда именно — зависит от ваших

настроек. В Ubuntu 16.04 они добавляются в журнал systemd, и посмотреть их можно вот так:

```
journalctl -u myapp.service
```

или в файле /var/log/syslog.

Можно сделать, чтобы systemd направлял вывод процесса в syslog, и написать правила обработки логов для него. Например сливать их на сервер-агрегатор логов или просто писать в файлы.

```
[Service]
Environment=MYAPP_PORT=3000
User=www-data
Group=www-data
ExecStart=/usr/local/bin/node /usr/local/www/myapp/index.js
Restart=always
# Отправка логов syslog
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=myapp
```

Опции StandardOutput и StandardError определяют, куда писать вывод приложения. Возможные значения этих опций вы можете посмотреть в man systemd.exec. SyslogIdentifier собственно задаёт имя сервиса для syslog. Кстати, при отправке логов в syslog, в журнал systemd они тоже будут писаться, так что можно по-прежнему читать их с помощью journalctl.

Ну так вот, в syslog можно настроить правила обработки логов от вашего приложения. У меня в качестве syslog-демона rsyslog. Его настройки лежат в /etc/rsyslog.d/. Создаю файл /etc/rsyslog.d/100-myapp.conf с таким содержимым:

```
$RepeatedMsgReduction off
if $programname == 'myapp' then -/var/log/myapp/debug.log
```

\$RepeatedMsgReduction off нужно, чтобы syslog несколько одинаковых сообщений не схлопывал в одно.

Systemd не умеет слать stdout и stderr в syslog с разным уровнем severity, поэтому в такой конфигурации нельзя писать stdout в один файл, а stderr в другой.

Можно направить stdout процесса в логгер (утилита для записи в syslog) пайпом, а stderr слать через systemd:

```
[Service]
Environment=MYAPP_PORT=3000
User=www-data
Group=www-data
# пайпим stdout в logger с тегом myapp
ExecStart=/bin/sh -c '/usr/local/bin/node /usr/local/www/myapp/index.js | logger
--tag myapp'
# stderr шлём в syslog средствами systemd
StandardError=syslog
SyslogIdentifier=myapp
# устанавливаем уровень логирования: err
SyslogLevel=err
Restart=always
```

И настроить в syslog правила для записи в разные файлы:

```
$RepeatedMsgReduction off
if $programname == 'myapp' and $syslogseverity < 5 then
-/var/log/myapp/error.log
if $programname == 'myapp' and $syslogseverity >= 5 then
-/var/log/myapp/debug.log
```

Кроме записи в файлы, можно конечно и на сервер-агрегатор слать.

Если вы пишете логи в файлы, не забудьте настроить logrotate. Пример файла конфигурации /etc/logrotate.d/myapp:

```
/var/log/myapp/debug.log
/var/log/myapp/error.log {
    rotate 7
    daily
    compress
    missingok
    notifempty
    postrotate
        invoke-rc.d rsyslog rotate > /dev/null
    endscript
}
```

Самое главное тут — скрипт `postrotate`, который заставляет `rsyslog` переоткрыть файл после ротации. Подробности про настройку `logrotate` можно прочитать в `man logrotate.conf`